

claude-windows / 188dd06f-9c99-487c-aa7d-b0bee83c629c

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\188dd06f-9c99-487c-aa7d-b0bee83c629c\subagents\workflows\wf_416cc0e5-a35\agent-a696791f7121f4e62.jsonl`
- SHA-256: `935c273f6629dc2e1de6f14ef1d4cb747462927c3ec32d8c7e48cf79569853e7`
- Source modified: `2026-06-22T09:51:49+00:00`
- Imported at: `2026-07-05T16:48:28+00:00`
- project: `wf_416cc0e5-a35`
- session_id: `188dd06f-9c99-487c-aa7d-b0bee83c629c`
```

Transcript

1. user (2026-06-22T09:49:37.810Z)

Review the NIGHTLY REEL-COUNT engine-regression change under C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-e2e (a NEW, additive PR). It runs the configured WASB pipeline on a GCP GPU VM nightly, asserts reel-count (==) + quality metrics + cost, emails a report, self-deletes. Files: scripts/nightly_reelcount.py, scripts/nightly_email.py, deploy/gcp/nightly_regression/run_on_vm.sh, deploy/gcp/nightly_regression/launch.sh, deploy/gcp/nightly_regression/register_scheduler.sh, deploy/gcp/nightly_regression/Dockerfile.launcher, eval_baselines/reelcount_manifest.json. Read them under that absolute path. Cite file:line. Report only REAL defects (correctness/safety/security that would bite in production or a regression that escapes), not style. Return raw structured findings.

ADVERSARIALLY VERIFY this finding ? confirm with concrete code evidence under C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-e2e or REFUTE it (default real=false if the evidence isn't clearly there). Finding:

```
{
  "title": "LOW: nightly_reelcount.py line 334 calls _load_gts(labels_uri) which may return None, then line 369 checks 'if gts and intervals'",
  "file": "C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-e2e/scripts/nightly_reelcount.py:272-292, 334, 368-372",
  "severity": "low",
  "detail": "Line 334 loads ground truth. _load_gts() returns None if labels_ref is None (line 276). Line 369 checks 'if gts and intervals' before calling score_intervals(), so missing labels safely skip quality scoring. This is correct ? quality metrics are optional. The manifest has labels_uri for each video, so this won't be hit in practice, but the graceful fallback is sound.",
  "fix": "No fix needed. The optional quality metrics are handled correctly."
}
```

2. assistant (2026-06-22T09:49:39.909Z)

I'll read and analyze the nightly regression engine files to verify the finding about the `\_load\_gts()` call and the None handling.

3. user (2026-06-22T09:49:41.287Z)

```
1      """Nightly END-TO-END engine regression ? reel-count + quality + cost (the GCP-VM
nightly).
2
3      Runs the **configured + checkpointed** pipeline (the real WASB GPU detector, as a
deployment serves
4      it) end-to-end on 2-3 short owned golden videos and asserts the **number of reels does
not change**
5      vs a frozen baseline. Unlike ``scripts/nightly_regression.py`` (PR #202 ? a CPU re-score
of CACHED
```

```

6      trajectories, the windowing/scorer only), this one runs the *full* pipeline on the *real
model*, so it
7      needs a GPU ? hence it runs on an ephemeral GCP GPU VM
(``deploy/gcp/nightly_regression/``), emails the
8      owner the report, and self-deletes. It is **default-additive**: a tripwire, not the
promotion gate.
9
10     What it asserts / reports, per the owner's ask ("quality metrics + regression report +
cost"):
11     ? REEL COUNT per video ? ``len(play_segments)`` after segmentation; the detected
highlight clips.
12     Pure-CV / WASB detection is deterministic for a fixed (video, config, weights), so
this is asserted
13     **EXACT** (with a re-run-once guard for the one known flake: a transient DB-insert-
None, see $guard).
14     ? QUALITY METRICS per video (when golden labels are provided) ? the R2 tolerance-match
block from
15     ``backend.eval.rally_seg_eval`` (tol_f1 / f1@0.5 / f1@0.25), compared with a small
tolerance.
16     ? COST of the run ? VM uptime (or processing wall) × machine $/hr via
``backend.billing`` ? USD + INR.
17
18     Extensible pattern (add a video = one manifest row + re-freeze):
19     ``eval_baselines/reelcount_manifest.json`` ? the videos (gs:// / local / gdrive://
URIs) + segmenter
20     ``eval_baselines/reelcount_baseline.json`` ? the frozen expected counts (regenerated,
never hand-edited)
21
22     Usage
23     -----
24     Freeze the baseline from the current configured+checkpointed engine (after an intended
change, or first run):
25     python scripts/nightly_reelcount.py --update-baseline
26
27     Nightly check (exit 1 on regression; writes output/nightly_reelcount/<date>.md; emails
if --email):
28     python scripts/nightly_reelcount.py --email
29
30     Exit codes: 0 pass · 1 regression (reel-count changed / a video errored / quality
dropped) · 2 operational
31     (manifest/videos missing, nothing ran) · 3 no baseline · 4 stale (corpus/segmenter
changed ? re-freeze).
32     """
33     from __future__ import annotations
34
35     import argparse
36     import datetime as _dt
37     import json
38     import os
39     import sys
40     from dataclasses import dataclass, field
41     from typing import Any, Dict, List, Optional, Tuple
42
43     REPO_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
44     if REPO_ROOT not in sys.path:
45         sys.path.insert(0, REPO_ROOT)
46
47     # billing is pure (typing only) ? safe to import at module top.
48     from backend import billing # noqa: E402
49
50     DEFAULT_MANIFEST = os.path.join(REPO_ROOT, "eval_baselines", "reelcount_manifest.json")
51     DEFAULT_BASELINE = os.path.join(REPO_ROOT, "eval_baselines", "reelcount_baseline.json")
52     DEFAULT_REPORT_DIR = os.path.join(REPO_ROOT, "output", "nightly_reelcount")
53
54     #: Quality metrics we report + (when labels exist) gate ? higher is better, small

```

```

tolerance.
55     QUALITY_HIGHER: Tuple[str, ...] = ("tol_f1", "f1@0.5", "f1@0.25")
56     DEFAULT_QUALITY_TOL = 0.02      # quality is float/labeled ? a looser tripwire than the
exact reel-count
57     DEFAULT_FX_INR_PER_USD = 83.0
58     #: Fallback machine price if the manifest doesn't pin one (T4 on n1-standard-8, asia-
south1, ~mid-2026).
59     DEFAULT_MACHINE_USD_PER_HR = 0.35
60
61
62     @dataclass(frozen=True)
63     class Tolerances:
64         quality_tol: float = DEFAULT_QUALITY_TOL
65
66         def to_dict(self) -> Dict[str, Any]:
67             return {"quality_tol": self.quality_tol}
68
69
70     # ----- #
71     # Pure cost math (wraps backend.billing ? no IO).
72     # ----- #
73     def cost_report(billed_seconds: float, machine_usd_per_hr: float,
74                     fx_inr_per_usd: float, gpu_seconds: float = 0.0) -> Dict[str, Any]:
75         """Run cost from billed wall-seconds × machine rate ? USD + INR (``backend.billing``
arithmetic).
76
77         ``billed_seconds`` is the VM uptime (boot?delete) when the orchestrator passes
``--vm-uptime-seconds``;
78         otherwise it's the in-process pipeline wall time (a lower bound ? the VM also pays
for boot/install)."""
79         rate_per_sec = float(machine_usd_per_hr) / 3600.0
80         usage = {billing.WALL_SECONDS: float(billed_seconds)}
81         rates = {billing.WALL_SECONDS: rate_per_sec}
82         usd, breakdown = billing.compute_cost_usd(usage, rates)
83         return {
84             "billed_seconds": round(float(billed_seconds), 1),
85             "gpu_seconds": round(float(gpu_seconds), 1),
86             "machine_usd_per_hr": float(machine_usd_per_hr),
87             "usd": usd,
88             "inr": billing.to_inr(usd, fx_inr_per_usd),
89             "breakdown": breakdown,
90         }
91
92
93     # ----- #
94     # Pure summary + comparison core (no IO ? unit-testable with plain dicts).
95     # ----- #
96     def summarize_results(run_results: List[Dict[str, Any]], segmenter: str,
97                           cost: Dict[str, Any]) -> Dict[str, Any]:
98         """Compact, comparable snapshot from per-video run results.
99
100         Each run result: ``{name, reel_count, ok, error, quality: {tol_f1,...}|None,
wall_seconds}``.
101         A video that errored keeps ``reel_count=None`` + ``ok=False`` (so the diff flags it,
never hides it)."""
102         per_video: Dict[str, Any] = {}
103         for r in run_results:
104             per_video[r["name"]] = {
105                 "reel_count": r.get("reel_count"),
106                 "ok": bool(r.get("ok", False)),
107                 "error": r.get("error"),
108                 "quality": r.get("quality"),
109             }
110         return {"segmenter": segmenter, "cost": cost, "per_video": per_video,
111               "n": len(per_video)}

```

```

112
113
114 @dataclass
115 class Finding:
116     video: str
117     metric: str          # "reel_count" | "error" | a quality key | "segmenter" |
"corpus"
118     kind: str           # "regression" | "improvement" | "stale"
119     baseline: Optional[float] = None
120     current: Optional[float] = None
121     detail: str = ""
122
123     def message(self) -> str:
124         if self.kind == "stale":
125             return f"[STALE] {self.metric}: {self.detail}"
126         if self.metric == "error":
127             return f"[REGRESSION] {self.video}: pipeline ERROR ? {self.detail}"
128         b = "?" if self.baseline is None else f"{self.baseline:g}"
129         c = "?" if self.current is None else f"{self.current:g}"
130         tag = "REGRESSION" if self.kind == "regression" else "improvement"
131         return f"[{tag}] {self.video}/{self.metric}: {b} ? {c}"
132
133
134 @dataclass
135 class NightlyResult:
136     findings: List[Finding] = field(default_factory=list)
137     added: List[str] = field(default_factory=list)
138     removed: List[str] = field(default_factory=list)
139
140     @property
141     def regressions(self) -> List[Finding]:
142         return [f for f in self.findings if f.kind == "regression"]
143
144     @property
145     def improvements(self) -> List[Finding]:
146         return [f for f in self.findings if f.kind == "improvement"]
147
148     @property
149     def stale(self) -> List[Finding]:
150         return [f for f in self.findings if f.kind == "stale"]
151
152     def overall_status(self) -> str:
153         if self.regressions:
154             return "REGRESSION"
155         if self.stale:
156             return "STALE"
157         return "PASS"
158
159     def exit_code(self) -> int:
160         if self.regressions:
161             return 1
162         if self.stale:
163             return 4
164         return 0
165
166
167     def compare(baseline: Dict[str, Any], current: Dict[str, Any], tols: Tolerances) ->
NightlyResult:
168         """Diff current run vs frozen baseline.
169
170         * Segmenter changed ? STALE (numbers not comparable; re-freeze).
171         * Corpus (video set) changed ? STALE + report added/removed; still diff the
INTERSECTION.
172         * Per shared video: reel_count EXACT (any change = regression); a pipeline ERROR =
regression;

```

```

173         quality metrics drop beyond ``quality_tol`` = regression (rise = improvement).
174         """
175         res = NightlyResult()
176         if baseline.get("segmenter") != current.get("segmenter"):
177             res.findings.append(Finding("", "segmenter", "stale",
178                                     detail=f"{baseline.get('segmenter')} ?
{current.get('segmenter')}}"))
179         return res
180
181         b_pv, c_pv = baseline.get("per_video", {}), current.get("per_video", {})
182         res.added = sorted(set(c_pv) - set(b_pv))
183         res.removed = sorted(set(b_pv) - set(c_pv))
184         if res.added or res.removed:
185             res.findings.append(Finding("", "corpus", "stale",
186                                     detail=f"added={res.added} removed={res.removed}"))
187
188         for v in sorted(set(b_pv) & set(c_pv)):
189             b, c = b_pv[v], c_pv[v]
190             # 1) pipeline must still succeed
191             if not c.get("ok", False) or c.get("reel_count") is None:
192                 res.findings.append(Finding(v, "error", "regression",
193                                     detail=str(c.get("error") or "no reel_count
produced")))
194                 continue
195             # 2) reel count ? EXACT
196             bc, cc = b.get("reel_count"), c.get("reel_count")
197             if bc is not None and cc is not None and cc != bc:
198                 res.findings.append(Finding(v, "reel_count", "regression", float(bc),
float(cc)))
199             # 3) quality metrics ? tolerance (only if both sides have them)
200             bq, cq = b.get("quality") or {}, c.get("quality") or {}
201             for m in QUALITY_HIGHER:
202                 bv, cv = bq.get(m), cq.get(m)
203                 if bv is None or cv is None:
204                     continue
205                 if cv < bv - tols.quality_tol:
206                     res.findings.append(Finding(v, m, "regression", bv, cv))
207                 elif cv > bv + tols.quality_tol:
208                     res.findings.append(Finding(v, m, "improvement", bv, cv))
209         return res
210
211
212         # ----- #
213         # Report formatting (pure).
214         # ----- #
215         def _q(qd: Optional[Dict[str, Any]], key: str) -> str:
216             if not qd or qd.get(key) is None:
217                 return "?"
218             return f"{qd[key]:.3f}"
219
220
221         def format_report(baseline: Dict[str, Any], current: Dict[str, Any],
222                           result: NightlyResult, date: str, head: str) -> str:
223             status = result.overall_status()
224             badge = {"PASS": "? PASS", "REGRESSION": "? REGRESSION",
225                     "STALE": "? STALE (re-freeze the baseline)"}[status]
226             cost = current.get("cost", {})
227             L: List[str] = [
228                 f"# Nightly engine regression (reel-count + quality + cost) ? {date}",
229                 "",
230                 f"***Status: {badge}** · exit code {result.exit_code()} · segmenter
`{current.get('segmenter')}`",
231                 "",
232                 f"- HEAD `{head}` · baseline `{baseline.get('git_commit', '?')}` (frozen
{baseline.get('generated_at', '?')})",

```

```

233         f"- **Run cost: ${cost.get('usd', 0):.4f} ({cost.get('inr', 0):.2f})** . "
234         f"billed {cost.get('billed_seconds', 0):.0f}s @ ${cost.get('machine_usd_per_hr',
0)}/hr",
235         "",
236     ]
237     if result.regressions:
238         L += ["## ? Regressions", ""] + [f"- {f.message()}" for f in result.regressions]
+ [""]
239     if result.stale:
240         L += ["## ? Stale / not comparable", ""] + [f"- {f.message()}" for f in
result.stale]
241         L += ["- Action: `python scripts/nightly_reelcount.py --update-baseline` (after
confirming the change is intended).", ""]
242
243     # Per-video table.
244     b_pv, c_pv = baseline.get("per_video", {}), current.get("per_video", {})
245     L += ["## Per-video", ""]
246         "| video | reels (base?now) | tol_f1 (base?now) | f1@0.5 | status |",
247         "|---|---|---|---|---|"
248     for v in sorted(set(b_pv) | set(c_pv)):
249         b, c = b_pv.get(v, {}), c_pv.get(v, {})
250         bc = b.get("reel_count")
251         cc = c.get("reel_count")
252         rc = f"'?' if bc is None else bc'?'{'ERROR' if not c.get('ok', True) else ('?'
if cc is None else cc)}"
253         bq, cq = b.get("quality"), c.get("quality")
254         tf = f"_{q(bq, 'tol_f1')}_?_{q(cq, 'tol_f1')}"
255         f5 = f"_{q(bq, 'f1@0.5')}_?_{q(cq, 'f1@0.5')}"
256         vstatus = "?" if any(f.video == v and f.kind == "regression" for f in
result.findings) else ""
257         L.append(f"| {v} | {rc} | {tf} | {f5} | {vstatus} |")
258     L.append("")
259
260     if result.improvements:
261         L += ["_Improvements over baseline (re-freeze to ratchet up):_" \
+ [f"- {f.message()}" for f in result.improvements] + [""]
262     L += ["---",
263         "_Full configured+checkpointed pipeline on the real model (GCP GPU VM).
Tripwire only ? does "
264         "not change the promotion gate. Complements PR #202's cached-trajectory CPU
re-score._"]
265     return "\n".join(L)
266
267
268
269     # ----- #
270     # IO shell ? running the configured pipeline per video (needs backend + GPU + video).
271     # ----- #
272     def _load_gts(labels_ref: Optional[str]) -> Optional[List[Tuple[float, float]]]:
273         """Load ground-truth rally intervals from a golden-features JSON (`gts` key) or a
rallies CSV
274         (`start,end` columns / pairs). Returns None when no labels are configured."""
275         if not labels_ref:
276             return None
277         from backend.storage import fetch
278         local = fetch(labels_ref)
279         if local.endswith(".json"):
280             with open(local, encoding="utf-8") as fh:
281                 data = json.load(fh)
282                 return [(float(s), float(e)) for s, e in (data.get("gts") or [])]
283         # CSV: header start,end (or first two numeric columns)
284         import csv
285         out: List[Tuple[float, float]] = []
286         with open(local, encoding="utf-8") as fh:
287             for row in csv.reader(fh):
288                 try:

```

```

289         out.append((float(row[0]), float(row[1])))
290     except (ValueError, IndexError):
291         continue
292     return out
293
294
295     def _segments_to_intervals(segs: List[Dict[str, Any]]) -> List[Tuple[float, float]]:
296         """(start, end) pairs from segmenter result dicts, tolerant of the two key
conventions in the
297         codebase (``start_time``/``end_time`` ? motion/DB shape ? or ``start``/``end``).
Used only for the
298         optional quality metrics; the reel COUNT is ``len(segs)`` regardless, so a key
mismatch degrades
299         quality scoring gracefully (skipped) without ever miscounting reels."""
300         out: List[Tuple[float, float]] = []
301         for s in segs:
302             start = s.get("start_time", s.get("start"))
303             end = s.get("end_time", s.get("end"))
304             if start is not None and end is not None:
305                 out.append((float(start), float(end)))
306         return out
307
308
309     def run_one_video(video: Dict[str, Any], segmenter: str, config: Any,
310                      rerun_on_mismatch: bool = True,
311                      baseline_count: Optional[int] = None) -> Dict[str, Any]:
312         """Run the configured pipeline on one video ? reel count (+ quality vs labels). Lazy
backend imports.
313
314         The re-run-once guard (the one known non-determinism ? a transient
``add_play_segment``?None drop,
315         database.py): if the count != the frozen baseline, re-process the video ONCE; a
transient drop won't
316         reproduce, a real change will."""
317         import random
318         import time
319
320         from backend.eval import rally_seg_eval
321         from backend.infrastructure.database import Database
322         from backend.pipeline.segmenters import segmenter_registry
323         from backend.results import Failure
324         from backend.storage import fetch
325         from backend.utils.hashing import compute_video_id
326
327         name = video["name"]
328         try:
329             local = fetch(video["uri"])
330         except Exception as e: # noqa: BLE001 ? surface, never hide
331             return {"name": name, "reel_count": None, "ok": False, "error": f"fetch failed:
{e}",
332                    "quality": None, "wall_seconds": 0.0}
333
334         gts = _load_gts(video.get("labels_uri"))
335         seg_cls = segmenter_registry.get(segmenter)
336         if not seg_cls:
337             return {"name": name, "reel_count": None, "ok": False,
338                    "error": f"segmenter '{segmenter}' not registered", "quality": None,
"wall_seconds": 0.0}
339
340     def _process(db_path: str) -> Tuple[Optional[int], Optional[List[Tuple[float,
float]]], float, Optional[str]]:
341         random.seed(0) # close motion.py's unseeded metadata RNG so secondary fields
are reproducible too
342         db = Database(db_path)
343         video_id = compute_video_id(local)

```

```

344         t0 = time.monotonic()
345         result = seg_cls(db, config).process_video(video_id, local)
346         wall = time.monotonic() - t0
347         if isinstance(result, Failure):
348             return None, None, wall, getattr(result, "message", "segmenter failed")
349         segs = result.value if hasattr(result, "value") else result
350         intervals = _segments_to_intervals(segs)
351         return len(segs), intervals, wall, None
352
353     import tempfile
354     total_wall = 0.0
355     with tempfile.TemporaryDirectory(prefix="nightly_reel_") as td:
356         count, intervals, wall, err = _process(os.path.join(td, "run1.db"))
357         total_wall += wall
358         if err is not None:
359             return {"name": name, "reel_count": None, "ok": False, "error": err,
360                    "quality": None, "wall_seconds": round(total_wall, 2)}
361         # re-run-once guard for the transient DB-drop flake
362         if rerun_on_mismatch and baseline_count is not None and count != baseline_count:
363             count2, intervals2, wall2, err2 = _process(os.path.join(td, "run2.db"))
364             total_wall += wall2
365             if err2 is None and count2 is not None:
366                 count, intervals = count2, intervals2 # the reproduced (or corrected)
367         value
368         quality = None
369         if gts and intervals is not None:
370             row = rally_seg_eval.score_intervals(intervals, gts)
371             quality = {k: row[k] for k in ("tol_f1", "f1@0.5", "f1@0.25",
372 "tol_precision", "tol_recall")}
373             if k in row}
374         return {"name": name, "reel_count": count, "ok": True, "error": None,
375                "quality": quality, "wall_seconds": round(total_wall, 2)}
376
377
378     def load_manifest(path: str) -> Dict[str, Any]:
379         with open(path, encoding="utf-8") as fh:
380             return json.load(fh)
381
382
383     def run_corpus(manifest: Dict[str, Any], baseline: Optional[Dict[str, Any]],
384                   rerun_on_mismatch: bool = True) -> Tuple[List[Dict[str, Any]], float]:
385         """Run every manifest video through the configured pipeline. Returns (run_results,
386 total_wall_s)."""
387         from backend.config import load_config
388         config = load_config()
389         # apply manifest config overrides onto the typed config (dotted keys)
390         for dotted, val in (manifest.get("config_overrides") or {}).items():
391             _set_dotted(config, dotted, val)
392         segmenter = manifest.get("segmenter") or getattr(getattr(config, "indexing", None),
393 "default_segmenter", "motion")
394         b_pv = (baseline or {}).get("per_video", {})
395         results: List[Dict[str, Any]] = []
396         total_wall = 0.0
397         for v in manifest.get("videos", []):
398             base_count = (b_pv.get(v["name"]) or {}).get("reel_count")
399             r = run_one_video(v, segmenter, config, rerun_on_mismatch, base_count)
400             total_wall += float(r.get("wall_seconds") or 0.0)
401             results.append(r)
402         return results, total_wall
403
404     def _set_dotted(config: Any, dotted: str, value: Any) -> None:

```



```

405     """Best-effort dotted-path set onto a pydantic config (e.g.
'indexing.motion_threshold')."""
406     parts = dotted.split(".")
407     obj = config
408     for p in parts[:-1]:
409         obj = getattr(obj, p, None)
410         if obj is None:
411             return
412     try:
413         setattr(obj, parts[-1], value)
414     except Exception: # noqa: BLE001 ? overrides are best-effort; a bad key shouldn't
crash the run
415         pass
416
417
418     def _git_head() -> str:
419         import subprocess
420         try:
421             out = subprocess.run(["git", "-C", REPO_ROOT, "rev-parse", "--short", "HEAD"],
422                                 capture_output=True, text=True, timeout=10)
423             return out.stdout.strip() or "?"
424         except Exception: # noqa: BLE001
425             return "?"
426
427
428     def load_baseline(path: str) -> Optional[Dict[str, Any]]:
429         if not os.path.exists(path):
430             return None
431         with open(path, encoding="utf-8") as fh:
432             return json.load(fh)
433
434
435     def save_baseline(path: str, snapshot: Dict[str, Any]) -> None:
436         out = dict(snapshot)
437         out["generated_at"] = _dt.date.today().isoformat()
438         out["git_commit"] = _git_head()
439         out["_doc"] = ("Frozen nightly reel-count baseline. Regenerate with `python
scripts/nightly_reelcount.py "
440                        "--update-baseline` after an INTENDED engine change or when adding a
video. Tied to the "
441                        "configured+checkpointed engine + the manifest corpus.")
442         out.pop("cost", None) # cost is per-run, not part of the frozen baseline
443         os.makedirs(os.path.dirname(path), exist_ok=True)
444         tmp = path + ".tmp"
445         with open(tmp, "w", encoding="utf-8") as fh:
446             json.dump(out, fh, indent=2, sort_keys=True)
447             fh.write("\n")
448         os.replace(tmp, path)
449
450
451     def build_parser() -> argparse.ArgumentParser:
452         p = argparse.ArgumentParser(description="Nightly E2E engine regression: reel-count +
quality + cost.")
453         p.add_argument("--manifest", default=DEFAULT_MANIFEST)
454         p.add_argument("--baseline", default=DEFAULT_BASELINE)
455         p.add_argument("--report-dir", default=DEFAULT_REPORT_DIR)
456         p.add_argument("--quality-tol", type=float, default=DEFAULT_QUALITY_TOL)
457         p.add_argument("--vm-uptime-seconds", type=float, default=None,
458                        help="authoritative VM uptime (boot?delete) for the cost line; "
459                        "defaults to the in-process pipeline wall time (a lower bound)")
460         p.add_argument("--no-rerun-guard", action="store_true",
461                        help="disable the re-run-once-on-mismatch guard (the DB-drop flake
mitigation)")
462         p.add_argument("--update-baseline", action="store_true",
463                        help="freeze the current reel counts + quality as the new baseline

```

```

instead of checking")
464     p.add_argument("--email", action="store_true", help="email the report (see
scripts/nightly_email.py)")
465     p.add_argument("--email-to", default=None, help="override the report recipient")
466     p.add_argument("--date", default=None, help="override the report date (YYYY-MM-DD;
for tests)")
467     p.add_argument("--no-report", action="store_true", help="skip writing the dated
report file")
468     return p
469
470
471     def main(argv: Optional[List[str]] = None) -> int:
472         import logging
473         logging.basicConfig(level=logging.INFO, format="%(levelname)s %(message)s")
474         log = logging.getLogger("nightly_reelcount")
475         args = build_parser().parse_args(argv)
476         tols = Tolerances(quality_tol=args.quality_tol)
477         date = args.date or _dt.date.today().isoformat()
478
479         if not os.path.exists(args.manifest):
480             log.error("manifest not found: %s", args.manifest)
481             return 2
482         manifest = load_manifest(args.manifest)
483         baseline = load_baseline(args.baseline)
484
485         try:
486             run_results, total_wall = run_corpus(manifest, baseline, rerun_on_mismatch=not
args.no_rerun_guard)
487             except Exception as e: # noqa: BLE001 ? operational failure (deps/config) ? exit 2,
never a false "pass"
488                 log.error("run failed: %s", e)
489                 return 2
490             if not run_results:
491                 log.error("no videos in manifest %s", args.manifest)
492                 return 2
493
494             billed = args.vm_uptime_seconds if args.vm_uptime_seconds is not None else
total_wall
495             cost = cost_report(
496                 billed_seconds=billed,
497                 machine_usd_per_hr=float(manifest.get("machine_usd_per_hr",
DEFAULT_MACHINE_USD_PER_HR)),
498                 fx_inr_per_usd=float(manifest.get("fx_inr_per_usd", DEFAULT_FX_INR_PER_USD)),
499                 gpu_seconds=total_wall,
500             )
501             segmenter = manifest.get("segmenter", "motion")
502             snapshot = summarize_results(run_results, segmenter, cost)
503             snapshot["git_commit"] = _git_head()
504             snapshot["generated_at"] = date
505
506             if args.update_baseline:
507                 save_baseline(args.baseline, snapshot)
508                 log.info("baseline frozen ? %s", args.baseline)
509                 for r in run_results:
510                     log.info(" %s: reels=%s ok=%s", r["name"], r.get("reel_count"),
r.get("ok"))
511                 return 0
512
513             if baseline is None:
514                 log.error("no baseline at %s ? freeze one with --update-baseline (on the
configured engine).", args.baseline)
515                 return 3
516
517             result = compare(baseline, snapshot, tols)
518             report = format_report(baseline, snapshot, result, date, snapshot["git_commit"])

```

```

519
520     if not args.no_report:
521         os.makedirs(args.report_dir, exist_ok=True)
522         md_path = os.path.join(args.report_dir, f"{date}.md")
523         with open(md_path, "w", encoding="utf-8") as fh:
524             fh.write(report + "\n")
525         with open(os.path.join(args.report_dir, f"{date}.json"), "w", encoding="utf-8")
as fh:
526             json.dump({"date": date, "status": result.overall_status(), "exit_code":
result.exit_code(),
527                       "snapshot": snapshot, "regressions": [f.message() for f in
result.regressions]},
528                       fh, indent=2, sort_keys=True)
529         log.info("report ? %s", md_path)
530
531     print(report)
532
533     if args.email:
534         try:
535             from scripts.nightly_email import send_report
536             subject = f"[KhelSutra nightly] {result.overall_status()} ? reel-count
engine regression {date}"
537             sent = send_report(subject, report, to=args.email_to)
538             log.info("email %s", "sent" if sent else "skipped (no creds ? report still
written)")
539         except Exception as e: # noqa: BLE001 ? a broken mailer must NOT mask the
regression verdict
540             log.error("email failed (report still written): %s", e)
541
542     code = result.exit_code()
543     if code == 1:
544         log.error("REEL-COUNT REGRESSION (%d) ? see %s", len(result.regressions),
args.report_dir)
545     elif code == 4:
546         log.warning("baseline STALE ? corpus/segmenter changed; re-freeze with --update-
baseline")
547     else:
548         log.info("nightly reel-count check PASSED (cost $%.4f / ?%.2f)", cost["usd"],
cost["inr"])
549     return code
550
551
552 if __name__ == "__main__":
553     raise SystemExit(main())
554

```

4. user (2026-06-22T09:49:41.539Z)

```

1      {
2      " _doc": "Nightly E2E engine reel-count manifest (scripts/nightly_reelcount.py). The
configured+checkpointed pipeline runs on each video on a GCP GPU VM; the reel count + quality
metrics are asserted vs eval_baselines/reelcount_baseline.json. ADD A VIDEO = append one row +
re-freeze: `python scripts/nightly_reelcount.py --update-baseline` (commit the updated
baseline). URIs may be gs:// (recommended, co-located with the VM), gdrive://, or local paths.
labels_uri (optional) is a golden-features JSON (gts key) or a start,end CSV ? present =>
quality metrics (tol_f1/f1@0.5/f1@0.25) are scored too. EDIT the example rows below to your
actual short (~5-6 min) OWNED, minors-free clips before enabling.",
3      "segmenter": "wasb_hybrid",
4      "detector_impl": "native",
5      "config_overrides": {
6          "indexing.default_segmenter": "wasb_hybrid",
7          "indexing.detector_impl": "native"
8      },
9      "machine_usd_per_hr": 0.70,
10     "fx_inr_per_usd": 83.0,

```

```

11     "videos": [
12         {
13             "name": "GX030094",
14             "uri": "gs://khealsutra-rally-corpus/nightly/videos/GX030094.mp4",
15             "labels_uri": "gs://khealsutra-rally-
corpus/nightly/labels/GX030094_golden_features.json"
16         },
17         {
18             "name": "Badminton_BXH_2",
19             "uri": "gs://khealsutra-rally-corpus/nightly/videos/Badminton_BXH_2.mp4",
20             "labels_uri": "gs://khealsutra-rally-
corpus/nightly/labels/Badminton_BXH_2_golden_features.json"
21         },
22         {
23             "name": "GX010128",
24             "uri": "gs://khealsutra-rally-corpus/nightly/videos/GX010128.mp4",
25             "labels_uri": "gs://khealsutra-rally-
corpus/nightly/labels/GX010128_golden_features.json"
26         }
27     ]
28 }
29

```

5. user (2026-06-22T09:49:42.277Z)

```

1     """Email the nightly regression report ? stdlib SMTP, creds from env or GCP Secret
Manager.
2
3     Used by ``scripts/nightly_reelcount.py --email`` (and reusable by any nightly).
Deliberately
4     dependency-light (stdlib ``smtplib`` + ``email.message``) and **graceful**: if no SMTP
password can
5     be resolved it logs + returns ``False`` (the report is still written/printed ? a broken
mailer must
6     never mask a regression).
7
8     Credential resolution (first hit wins), so it works on a GCP VM, locally, or in tests:
9     ? Recipient: ``--email-to`` arg ? ``NIGHTLY_EMAIL_TO`` ? ``avi.dullu@gmail.com``
10    ? SMTP host: ``NIGHTLY_SMTP_HOST`` (default ``smtp.gmail.com``) . port
``NIGHTLY_SMTP_PORT`` (587, STARTTLS)
11    ? SMTP user: ``NIGHTLY_SMTP_USER`` (default = recipient)
12    ? SMTP pass: ``NIGHTLY_SMTP_PASS`` ? Secret Manager ``NIGHTLY_SMTP_SECRET``
13    (``projects/<p>/secrets/<name>/versions/latest``; via the google-cloud-
secret-manager
14    lib if installed, else the ``gcloud`` CLI). A Gmail **app password** is
the simplest.
15
16    Set the secret once (owner): echo -n '<app-password>' | gcloud secrets create nightly-
smtp-pass --data-file=-
17    and on the VM: export NIGHTLY_SMTP_SECRET=projects/<proj>/secrets/nightly-smtp-
pass/versions/latest
18    export NIGHTLY_SMTP_USER=<your-gmail>
NIGHTLY_EMAIL_TO=avi.dullu@gmail.com
19    """
20    from __future__ import annotations
21
22    import logging
23    import os
24    from email.message import EmailMessage
25    from typing import Optional
26
27    log = logging.getLogger("nightly_email")
28
29    DEFAULT_TO = "avi.dullu@gmail.com"
30    DEFAULT_HOST = "smtp.gmail.com"

```

```

31     DEFAULT_PORT = 587
32
33
34     def build_message(subject: str, body_md: str, from_addr: str, to_addr: str) ->
EmailMessage:
35         """A text/plain email carrying the markdown report (renders fine in any client).
Pure ? testable."""
36         msg = EmailMessage()
37         msg["Subject"] = subject
38         msg["From"] = from_addr
39         msg["To"] = to_addr
40         msg.set_content(body_md)
41         return msg
42
43
44     def resolve_secret(env: Optional[dict] = None) -> Optional[str]:
45         """Resolve the SMTP password from env or GCP Secret Manager. Returns None if
unresolvable."""
46         env = env if env is not None else os.environ
47         direct = env.get("NIGHTLY_SMTP_PASS")
48         if direct:
49             return direct
50         secret_ref = env.get("NIGHTLY_SMTP_SECRET")
51         if not secret_ref:
52             return None
53         # Prefer the client lib; fall back to the gcloud CLI (no extra dependency required).
54         try:
55             from google.cloud import secretmanager # type: ignore
56             client = secretmanager.SecretManagerServiceClient()
57             return
client.access_secret_version(name=secret_ref).payload.data.decode("utf-8")
58         except Exception: # noqa: BLE001 ? lib absent or auth issue ? try the CLI
59             pass
60         try:
61             import subprocess
62             name = secret_ref.split("/secrets/", 1)[1].split("/", 1)[0] if "/secrets/" in
secret_ref else secret_ref
63             out = subprocess.run(["gcloud", "secrets", "versions", "access", "latest", "--
secret", name],
64                                 capture_output=True, text=True, timeout=30)
65             return out.stdout.strip() if out.returncode == 0 and out.stdout.strip() else
None
66         except Exception as e: # noqa: BLE001
67             log.warning("secret-manager resolution failed: %s", e)
68             return None
69
70
71     def send_report(subject: str, body_md: str, to: Optional[str] = None,
72                   env: Optional[dict] = None) -> bool:
73         """Send the report. Returns True if sent, False if skipped (no creds) ? never raises
on missing creds."""
74         env = env if env is not None else os.environ
75         to_addr = to or env.get("NIGHTLY_EMAIL_TO") or DEFAULT_TO
76         host = env.get("NIGHTLY_SMTP_HOST", DEFAULT_HOST)
77         port = int(env.get("NIGHTLY_SMTP_PORT", DEFAULT_PORT))
78         user = env.get("NIGHTLY_SMTP_USER") or to_addr
79         password = resolve_secret(env)
80         if not password:
81             log.warning("no SMTP password (set NIGHTLY_SMTP_PASS or NIGHTLY_SMTP_SECRET) ?
email skipped.")
82             return False
83
84         import smtplib
85         msg = build_message(subject, body_md, from_addr=user, to_addr=to_addr)
86         with smtplib.SMTP(host, port, timeout=60) as s:

```

```

87         s.starttls()
88         s.login(user, password)
89         s.send_message(msg)
90         log.info("emailed report to %s", to_addr)
91         return True
92

```

6. assistant (2026-06-22T09:49:44.419Z)

Now let me search for where the finding's claimed issue arises ? I need to examine the specific lines mentioned:

7. user (2026-06-22T09:49:46.389Z)

```

270     # IO shell ? running the configured pipeline per video (needs backend + GPU + video).
271     # ----- #
272     def _load_gts(labels_ref: Optional[str]) -> Optional[List[Tuple[float, float]]]:
273         """Load ground-truth rally intervals from a golden-features JSON (`gts` key) or a
rallies CSV
274         (`start,end` columns / pairs). Returns None when no labels are configured."""
275         if not labels_ref:
276             return None
277         from backend.storage import fetch
278         local = fetch(labels_ref)
279         if local.endswith(".json"):
280             with open(local, encoding="utf-8") as fh:
281                 data = json.load(fh)
282                 return [(float(s), float(e)) for s, e in (data.get("gts") or [])]
283         # CSV: header start,end (or first two numeric columns)
284         import csv
285         out: List[Tuple[float, float]] = []
286         with open(local, encoding="utf-8") as fh:
287             for row in csv.reader(fh):
288                 try:
289                     out.append((float(row[0]), float(row[1])))
290                 except (ValueError, IndexError):
291                     continue
292         return out
293
294
295     def _segments_to_intervals(segs: List[Dict[str, Any]]) -> List[Tuple[float, float]]:
296         """(`start, end` pairs from segmenter result dicts, tolerant of the two key
conventions in the
297         codebase (`start_time`/`end_time` ? motion/DB shape ? or `start`/`end`).
Used only for the
298         optional quality metrics; the reel COUNT is `len(segs)` regardless, so a key
mismatch degrades
299         quality scoring gracefully (skipped) without ever miscounting reels."""
300         out: List[Tuple[float, float]] = []
301         for s in segs:
302             start = s.get("start_time", s.get("start"))
303             end = s.get("end_time", s.get("end"))
304             if start is not None and end is not None:
305                 out.append((float(start), float(end)))
306         return out
307
308
309     def run_one_video(video: Dict[str, Any], segmenter: str, config: Any,
310                      rerun_on_mismatch: bool = True,
311                      baseline_count: Optional[int] = None) -> Dict[str, Any]:
312         """Run the configured pipeline on one video ? reel count (+ quality vs labels). Lazy
backend imports.
313
314         The re-run-once guard (the one known non-determinism ? a transient
`add_play_segment`?None drop,

```

```

315         database.py): if the count != the frozen baseline, re-process the video ONCE; a
transient drop won't
316         reproduce, a real change will."""
317         import random
318         import time
319
320         from backend.eval import rally_seg_eval
321         from backend.infrastructure.database import Database
322         from backend.pipeline.segmenters import segmenter_registry
323         from backend.results import Failure
324         from backend.storage import fetch
325         from backend.utils.hashing import compute_video_id
326
327         name = video["name"]
328         try:
329             local = fetch(video["uri"])
330         except Exception as e: # noqa: BLE001 ? surface, never hide
331             return {"name": name, "reel_count": None, "ok": False, "error": f"fetch failed:
{e}",
332                     "quality": None, "wall_seconds": 0.0}
333
334         gts = _load_gts(video.get("labels_uri"))
335         seg_cls = segmenter_registry.get(segmenter)
336         if not seg_cls:
337             return {"name": name, "reel_count": None, "ok": False,
338                     "error": f"segmenter '{segmenter}' not registered", "quality": None,
"wall_seconds": 0.0}
339
340         def _process(db_path: str) -> Tuple[Optional[int], Optional[List[Tuple[float,
float]]], float, Optional[str]]:
341             random.seed(0) # close motion.py's unseeded metadata RNG so secondary fields
are reproducible too
342             db = Database(db_path)
343             video_id = compute_video_id(local)
344             t0 = time.monotonic()
345             result = seg_cls(db, config).process_video(video_id, local)
346             wall = time.monotonic() - t0
347             if isinstance(result, Failure):
348                 return None, None, wall, getattr(result, "message", "segmenter failed")
349             segs = result.value if hasattr(result, "value") else result
350             intervals = _segments_to_intervals(segs)
351             return len(segs), intervals, wall, None
352
353         import tempfile
354         total_wall = 0.0
355         with tempfile.TemporaryDirectory(prefix="nightly_reel_") as td:
356             count, intervals, wall, err = _process(os.path.join(td, "run1.db"))
357             total_wall += wall
358             if err is not None:
359                 return {"name": name, "reel_count": None, "ok": False, "error": err,
360                         "quality": None, "wall_seconds": round(total_wall, 2)}
361             # re-run-once guard for the transient DB-drop flake
362             if rerun_on_mismatch and baseline_count is not None and count != baseline_count:
363                 count2, intervals2, wall2, err2 = _process(os.path.join(td, "run2.db"))
364                 total_wall += wall2
365                 if err2 is None and count2 is not None:
366                     count, intervals = count2, intervals2 # the reproduced (or corrected)
value
367
368             quality = None
369             if gts and intervals is not None:
370                 row = rally_seg_eval.score_intervals(intervals, gts)
371                 quality = {k: row[k] for k in ("tol_f1", "f1@0.5", "f1@0.25",
"tol_precision", "tol_recall")}
372                 if k in row}

```

```
373
374         return {"name": name, "reel_count": count, "ok": True, "error": None,
```

8. user (2026-06-22T09:49:47.628Z)

```
689c738 docs(pkg): mark P2c ? + the loop closed ? v1 LIGHT is feature-complete (#313)
293e5c0 feat(pkg): re-bundle backend/ui ? serve the P2c first-run wizard from indexer --app
(#312)
f8b2c3b feat(pkg): P2b-install ? batteries-included install + truly-local 'motion' first-run
default (gate 08) (#310)
0ed4cc6 fix(segmenter): fail the job on an unrunnable config instead of a silent 0-rally success
(Finding B) (#311)
a42c132 docs(pkg): track the batteries-included app plan (PACKAGING_PLAN.md) + index pointers
(#309)
de2bf14 feat(pkg): P2e ? lazy-torch so the LIGHT tier imports with NO torch installed (default-
OFF) (#308)
679e642 feat(pkg): P2d ? DB upgrade-safety (downgrade guard + backup-before-migrate + export)
(default-OFF) (#307)
f01dc85 feat(db): v7 ? record the UI locale per job (PMF analytics) + count_jobs_by_locale
(#306)
fa7ee3f feat(pkg): P2a ? single ffmpeg/ffprobe resolver (env-overridable; one fail-loud place)
(default-OFF) (#305)
e99d0c3 feat(stitch): localize the reel watermark to the UI locale (+ OFL Noto fonts) (#304)
ca34787 feat(pkg): P2b ? friendly `indexer --app` launcher (boot localhost server + open the
bundled UI) (#303)
4a06b8b feat(pkg): P1b ? bundle the KhelSutra SPA single-origin into the engine wheel (default-
OFF) (#302)
be0c434 feat(pkg): P1a ? net-new console/wizard endpoints: /api/version, /api/capabilities,
/api/videos, /api/reels (#300)
cb7394b feat(serving): T11 ? retention sweep for stale uploads + caches (#299)
c039b90 feat(pkg): P0e ? single-instance lock so a 2nd process can't corrupt in-flight job state
(default-OFF) (#298)
e8bad40 feat(pkg): P0d ? declare scipy (undeclared serving dep) + license-scan & built-wheel CI
lanes (#297)
3895963 feat(pkg): P0c ? config/profile-driven bind address + DNS-rebinding Host-header guard
(default-OFF) (#296)
fc78fce feat(pkg): P0b ? redact secret-shaped fields from the unauthenticated GET /api/config
(#295)
0c0e064 feat(serving): exposure-safety boot posture ? fail fast on a half-configured exposed
engine (default-OFF) (#294)
82a28f4 feat(pkg): P0a ? resolved app-data HOME (RALLY_APP_HOME) for config/.env/DB/output
(default-OFF) (#293)
```

9. assistant (2026-06-22T09:49:54.508Z)

Perfect. Now I have the key code sections. Let me trace through the exact flow:

```
**Lines 272?292:** `_load_gts()` function
- Line 275?276: Returns `None` if `labels_ref` is falsy
- Lines 277?292: Otherwise fetches and parses the labels

**Line 334:** Calls `_load_gts(video.get("labels_uri"))` and assigns to `gts`
- `gts` can be `None` (if `labels_uri` is absent or falsy) or a list of tuples

**Lines 368?372:** Quality metric scoring
```python
quality = None
if gts and intervals is not None:
 row = rally_seg_eval.score_intervals(intervals, gts)
 quality = {k: row[k] for k in ("tol_f1", "f1@0.5", "f1@0.25", "tol_precision", "tol_recall")}
 if k in row}
...`
```

Let me also check the manifest to verify the assumption that labels\_uri is always present:



#### 10. user (2026-06-22T09:49:55.647Z)

Found 2 files  
scripts\nightly\_reelcount.py  
eval\_baselines\reelcount\_manifest.json

#### 11. assistant (2026-06-22T09:49:59.700Z)

Now let me verify the actual manifest JSON to confirm all three videos have `labels\_uri`:

Looking back at the manifest I already read (lines 1-28 above), I can confirm:

```
- **Line 15:** GX030094 has `"labels_uri": "gs://...GX030094_golden_features.json"`
- **Line 20:** Badminton_BXH_2 has `"labels_uri":
"gs://...Badminton_BXH_2_golden_features.json"`
- **Line 25:** GX010128 has `"labels_uri": "gs://...GX010128_golden_features.json"`
```

Now let me examine the control flow more carefully to verify the finding:

#### 12. user (2026-06-22T09:50:01.166Z)

```
369: if gts and intervals is not None:
```

#### 13. assistant (2026-06-22T09:50:22.842Z)

Perfect. Now let me verify the complete data flow by checking what happens after line 334 and what `intervals` could be:

Looking at the code:

```
1. **Line 334:** `gts = _load_gts(video.get("labels_uri"))`
 - Returns `None` if `labels_uri` is missing or `None`
 - Returns `None` if `labels_uri` is falsy/empty string
 - Otherwise returns a list of `(float, float)` tuples

2. **Line 356:** `count, intervals, wall, err = _process(os.path.join(td, "run1.db"))`
 - `_process()` always returns `(count, intervals, wall, err)` tuple
 - On error (line 348): returns `(None, None, wall, error_msg)`
 - On success (line 351): returns `(len(segs), intervals, wall, None)`

3. **Line 358?360:** If `err` is not `None`, returns early with `quality: None`

4. **Lines 362?366:** Re-run guard
 - Only executed if no error on first run
 - Updates `intervals` if the second run succeeds

5. **Lines 368?372:** Quality scoring
   ```python  
   quality = None  
   if gts and intervals is not None:  
       row = rally_seg_eval.score_intervals(intervals, gts)  
       quality = {...}  
   ```
```

Now let me verify what happens when the condition is not met:

The condition `if gts and intervals is not None:` means:

```
- If `gts` is `None` (no labels configured) ? quality stays `None`
- If `intervals` is `None` (shouldn't happen unless error, which is caught earlier) ? quality stays `None`
- Otherwise, score the intervals
```

Let me double-check that `intervals` can only be `None` if there was an error:

Looking at `\_process()` (lines 340?351):

```
- Line 348: If segmenter fails (Failure result), returns `(None, None, wall, error_msg)`
```

- Line 350: Otherwise, `intervals = \_segments\_to\_intervals(segs)` which always returns a list (possibly empty)
- Line 351: Returns the tuple with `intervals` (which is a list)

So `intervals` is `None` **only** when there's an error in `\_process()`, and that error is caught at line 358.

**The finding claims:** Line 334 calls `\_load\_gts(labels\_uri)` which may return None, then line 369 checks `if gts and intervals` ? and this is correct handling.

Let me verify the actual claim against the code one more time to ensure I understand what's being asserted:

The finding states:

> "Line 334 loads ground truth. `\_load\_gts()` returns None if labels\_ref is None (line 276). Line 369 checks 'if gts and intervals' before calling score\_intervals(), so missing labels safely skip quality scoring. This is correct ? quality metrics are optional."

**VERIFICATION:**

1. **Line 334 loads GTS:** ? `gts = \_load\_gts(video.get("labels\_uri"))`
2. **`\_load\_gts()` returns None if labels\_ref is None:** ? Lines 275?276
3. **Line 369 checks both conditions:** ? `if gts and intervals is not None:`
4. **Missing labels safely skip quality scoring:** ? When `gts` is falsy, `quality` stays `None` (line 368)
5. **Manifest has labels\_uri for each video:** ? All three videos in manifest have labels\_uri
6. **This won't be hit in practice:** ? True ? manifest ensures labels are present
7. **Graceful fallback is sound:** ? The None check allows optional labels

Now let me check if there's any scenario where this could be a real defect by looking at how `score\_intervals()` handles its inputs:

#### 14. user (2026-06-22T09:50:24.153Z)

```
./backend/eval/eval_partial_labels.py
./backend/eval/eval_stats.py
./backend/eval/per_user_eval.py
./backend/eval/rally_seg_eval.py
./backend/eval/__pycache__/eval_partial_labels.cpython-313.pyc
./backend/eval/__pycache__/eval_stats.cpython-313.pyc
./backend/eval/__pycache__/per_user_eval.cpython-313.pyc
./backend/eval/__pycache__/rally_seg_eval.cpython-313.pyc
./run_eval.py
./scripts/run_phase3_eval.py
./tests/test_eval_annotations.py
./tests/test_eval_batch.py
./tests/test_eval_classifier.py
./tests/test_eval_harness.py
./tests/test_eval_metrics.py
./tests/test_eval_partial_labels.py
./tests/test_eval_regression.py
./tests/test_eval_stats.py
./tests/test_eval_training.py
./tests/test_per_user_eval.py
```

#### 15. user (2026-06-22T09:50:32.984Z)

Found 1 file  
backend\eval\rally\_seg\_eval.py

#### 16. user (2026-06-22T09:50:38.053Z)

```
1 """One-command rally-segmentation eval ? score a windowing against the golden labels
with the
```

```

2 over-segmentation-sensitive metric set, honestly.
3
4 This is the harness the rally-quality project (`docs/RALLY_QUALITY_RESEARCH.md` $4.5 /
5 $7)
6 measures every tier against. It closes the loop the over-merge finding exposed: plain
7 temporal
8 IoU-F1 is blind to merged mega-windows, so we report it **alongside** segmental F1@k,
9 the
10 segment-count ratio, and the explicit `merge_split` breakdown
11 (`segmentation_metrics`).
12
13 For each golden video it: parses the cached WASB/TrackNet trajectory CSV ? builds
14 candidate
15 windows under a :class:`~backend.eval.windowing.WindowingPreset` (so it tracks the
16 SERVED
17 operating point and any future windowing the same way) ? scores the windows-as-rallies
18 against
19 the ground-truth rally intervals. Aggregates per-video held-out F1 with a bootstrap CI,
20 and can
21 diff two windowings with a paired sign-test + CI (the per-tier `delta`).
22
23 GROUND TRUTH: the golden manifest (`output/human_lovo_manifest.json`) gives each
24 video's
25 trajectory path + fps + frame_width; the GT rally intervals come from
26 `output/<name>_golden_features.json` (the `gts` key) so this needs no extra label
27 files.
28
29 Pure offline (CPU): trajectory parsing + windowing + interval scoring; no GPU/Gemini.
30 """
31 from __future__ import annotations
32
33 import argparse
34 import json
35 import os
36 from typing import Any, Dict, List, Optional, Tuple
37
38 from backend.eval import eval_stats, metrics, segmentation_metrics, windowing
39 from backend.eval.metrics import Interval
40 from backend.eval.windowing import PRESETS, SERVED, WindowingPreset
41
42 DEFAULT_MANIFEST = os.path.join("output", "human_lovo_manifest.json")
43 DEFAULT_FEATURES_DIR = "output"
44 DEFAULT_IOU = 0.5
45
46 #: Long-rally lens threshold (s). A rally is "long" (an eye-catching highlight) when its
47 duration
48 #: (end ? start) exceeds this. The local detector over-fires, and its false positives
49 are mostly
50 #: SHORT (motion blips, background-court fragments on multicourt footage); filtering to
51 long rallies
52 #: strips most of those FPs and reveals the real product quality on the long-rally happy
53 path. Used
54 #: as the default split point for the stratified report (`--long-tau` overrides it).
55 DURATION, not
56 #: shot-count, is the filter: golden `shots_count` is unlabeled and the no-AI path
57 reports it as
58 #: "Unknown", whereas duration is derived from the same start/end labels we already
59 trust.
60 LONG_RALLY_TAU = 5.0
61
62 #: Multicourt golden clips ? the court-type lookup for the stratified report. The golden
63 manifest
64 #: (`output/human_lovo_manifest.json`) is gitignored, so this committed set is the
65 AUDITABLE source
66 #: of the single-vs-multicourt strata, kept in sync with `docs/GOLDEN_VIDEOS.md` (the

```

```

clips tagged
48 #: "multicourt"). Multicourt footage carries background games on adjacent courts ? the
dominant
49 #: source of short false positives ? so we report it separately. A manifest entry MAY
carry an
50 #: explicit ``court_type`` field, which overrides this set (see :func:`court_type_for`).
51 MULTICOURT_CLIPS = frozenset({"mahadevpura_2", "GX010128", "Badminton_BXH_2",
"Boxhill_Doubles"})
52
53
54 def court_type_for(video: Dict[str, Any]) -> str:
55 """``multicourt`` or ``single`` for a golden video. Honors an explicit manifest
56 ``court_type`` field when present, else falls back to membership in
``MULTICOURT_CLIPS``
57 (keyed by ``name``), else ``single``."""
58 ct = video.get("court_type")
59 if isinstance(ct, str) and ct:
60 return ct
61 return "multicourt" if video.get("name") in MULTICOURT_CLIPS else "single"
62
63
64 def filter_by_min_duration(ivs: List[Interval], min_duration: float) -> List[Interval]:
65 """The long-rally lens: keep only intervals strictly longer than ``min_duration``
seconds.
66
67 ``min_duration <= 0`` is an EXACT passthrough (the default-OFF semantics ? every
real rally has
68 positive duration, so nothing is dropped). It's a FILTER, not a new metric: applied
identically
69 to predictions AND ground truth before any matching, so the tIoU and R2 paths stay
consistent."""
70 if min_duration <= 0.0:
71 return list(ivs)
72 return [iv for iv in ivs if (iv[1] - iv[0]) > min_duration]
73
74
75 # ----- #
76 # Pure scoring core (no I/O ? unit-testable)
77 # ----- #
78 def score_intervals(preds: List[Interval], gts: List[Interval],
79 iou: float = DEFAULT_IOU,
80 tau_start: Optional[float] = None,
81 tau_end: Optional[float] = None,
82 min_duration: float = 0.0) -> Dict[str, Any]:
83 """The full per-video metric row for predicted vs ground-truth rally intervals.
84
85 Pairs temporal-IoU P/R/F1 + boundary error (``metrics.score``) with the
86 over-segmentation-sensitive set (F1@k, segment-count ratio, merge/split breakdown)
so a
87 merged mega-window is *visible* even when IoU-F1 isn't moved.
88
89 Also carries the **R2** task-faithful block (``tolerance_metrics``, the headline
going forward;
90 tIoU here is the SECONDARY trend-line per the augment-then-ablation-gate decision):
strict-1:1
91 tolerance-match ``tol_f1/tol_precision/tol_recall`` (over-production costs
precision) + the
92 start/end boundary-error medians + the over-seg guard counts. See
docs/R2_EVAL_METRIC_DESIGN.md.
93
94 ``min_duration`` > 0 applies the **long-rally lens**
(:func:`filter_by_min_duration`): BOTH preds
95 and GTs are filtered to rallies longer than that many seconds BEFORE any matching,
so the whole
96 row (tIoU set AND R2 block) scores only long rallies. Default 0.0 = OFF (exact

```

```

passthrough).
97 """
98 from backend.eval import tolerance_metrics as tm
99 preds = filter_by_min_duration(preds, min_duration)
100 gts = filter_by_min_duration(gts, min_duration)

```

## 17. assistant (2026-06-22T09:50:40.591Z)

Good! Let me see more of this function to confirm it handles the inputs correctly:

## 18. user (2026-06-22T09:50:42.172Z)

```

98 from backend.eval import tolerance_metrics as tm
99 preds = filter_by_min_duration(preds, min_duration)
100 gts = filter_by_min_duration(gts, min_duration)
101 ts = tm.TAU_START if tau_start is None else tau_start
102 te = tm.TAU_END if tau_end is None else tau_end
103 sc = metrics.score(preds, gts, iou_threshold=iou)
104 f1k = segmentation_metrics.f1_at_overlaps(preds, gts)
105 ms = segmentation_metrics.merge_split_report(preds, gts)
106 tol_matches, tol_p, tol_r, tol_f1 = tm.match_1to1(preds, gts, ts, te)
107 be = tm.boundary_error_report(preds, gts) # unconditional best-overlap (the
R4-aiming diagnostic)
108 osr = tm.over_seg_report(preds, gts)
109 return {
110 "f1": sc.f1, "precision": sc.precision, "recall": sc.recall, "mean_iou":
sc.mean_iou,
111 "mean_start_error": sc.mean_start_error, "mean_end_error": sc.mean_end_error,
112 "f1@0.1": f1k[0.1], "f1@0.25": f1k[0.25], "f1@0.5": f1k[0.5],
113 "segment_count_ratio": segmentation_metrics.segment_count_ratio(preds, gts),
114 "merge_rate": ms["merge_rate"], "split_rate": ms["split_rate"],
115 "merges": ms["merges"], "splits": ms["splits"], "missed": ms["missed"],
116 "spurious": ms["spurious"], "num_pred": len(preds), "num_gt": len(gts),
117 # --- R2 (tolerance-match) ? the task-faithful headline block ---
118 "tau_start": ts, "tau_end": te,
119 "tol_f1": tol_f1, "tol_precision": tol_p, "tol_recall": tol_r,
120 "tol_matched": float(len(tol_matches)),
121 "dstart_abs_median": be["dstart"]["abs_median"], "dend_abs_median":
be["dend"]["abs_median"],
122 "split_count": osr["split_count"], "merge_count": osr["merge_count"],
123 }
124
125
126 _AGG_KEYS = ("f1", "f1@0.25", "f1@0.5", "merge_rate", "split_rate",
"segment_count_ratio",
127 "precision", "recall",
128 "tol_f1", "tol_precision", "tol_recall", "dstart_abs_median",
"dend_abs_median")
129
130
131 def aggregate(rows: List[Dict[str, Any]]) -> Dict[str, Any]:
132 """Aggregate per-video rows: mean (+ bootstrap 95% CI) of the headline metrics
across videos.
133
134 Per-video means (not pooled) keep videos the unit of evidence, matching the
harness's
135 leave-one-video-out discipline (windows within a video are correlated)."""
136 out: Dict[str, Any] = {"n": len(rows)}
137 for k in _AGG_KEYS:
138 vals = [r[k] for r in rows]
139 out[k] = eval_stats.mean_with_ci(vals) if vals else None
140 return out
141
142
143 # ----- #

```

```

144 # Golden corpus I/O
145 # ----- #
146 def load_golden(manifest_path: str = DEFAULT_MANIFEST,
147 features_dir: str = DEFAULT_FEATURES_DIR) -> List[Dict[str, Any]]:

```

#### 19. assistant (2026-06-22T09:50:49.107Z)

Perfect! The `score_intervals()` function signature and implementation shows:

- **Line 78:** Takes `preds` and `gts` as list parameters
- **Line 99-100:** Filters both lists by duration
- **Line 103-108:** Calls various scoring functions on the filtered lists
- **Line 109-123:** Returns a dict with all the keys including `"tol_f1"`, `"f1@0.5"`, `"f1@0.25"` etc.

The function assumes both `preds` and `gts` are valid lists. If `gts` were `None`, line 100 would fail with `TypeError` when trying to call `filter_by_min_duration(None, ...)`.

This is exactly why the check at line 369 is **critical and correct**:

```

```python
if gts and intervals is not None:
    row = rally_seg_eval.score_intervals(intervals, gts)
...

```

The condition prevents calling `score_intervals()` with `None` for `gts`.

Now let me verify one more thing ? are there any test cases or other code paths that show this scenario actually being tested?

20. user (2026-06-22T09:50:51.157Z)

(Bash completed with no output)

21. user (2026-06-22T09:50:54.203Z)

```

tests/test_nightly_reelcount.py
tests/test_nightly_regression.py
tests/__pycache__/test_nightly_reelcount.cpython-313-pytest-9.0.3.pyc
tests/__pycache__/test_nightly_regression.cpython-313-pytest-9.0.3.pyc

```

22. user (2026-06-22T09:50:56.545Z)

```

1     """Tests for scripts/nightly_reelcount.py + scripts/nightly_email.py ? the pure cores.
2
3     Fully offline: no GPU, no video, no network, no real corpus. The comparison / cost /
report / email
4     formatting operate on plain dicts + strings, so reel-count regressions, quality drops,
pipeline
5     errors, corpus drift, and the no-creds email skip are all exercised deterministically.
The actual
6     pipeline run (run_one_video / run_corpus) needs a video + GPU and is owner-verified on
the VM.
7     """
8     import importlib.util
9     import os
10    import sys
11
12    import pytest
13
14    _SCRIPTS = os.path.join(os.path.dirname(os.path.dirname(os.path.abspath(__file__))),
"scripts")
15
16
17    def _load(modname):
18        spec = importlib.util.spec_from_file_location(modname, os.path.join(_SCRIPTS,

```

```

f"{modname}.py"))
19     mod = importlib.util.module_from_spec(spec)
20     sys.modules[modname] = mod # register before exec so dataclass annotations resolve
21     spec.loader.exec_module(mod)
22     return mod
23
24
25     nr = _load("nightly_reelcount")
26     ne = _load("nightly_email")
27
28
29     # ----- #
30     # Builders
31     # ----- #
32     def _video(name, reel_count, ok=True, error=None, quality=None):
33         return {"name": name, "reel_count": reel_count, "ok": ok, "error": error,
34                "quality": quality, "wall_seconds": 1.0}
35
36
37     def _snap(per_video, segmenter="wasb_hybrid"):
38         return {"segmenter": segmenter, "per_video": per_video,
39                "cost": {"usd": 0.05, "inr": 4.15, "billed_seconds": 500,
"machine_usd_per_hr": 0.35}}
40
41
42     TOLS = nr.Tolerances()
43
44
45     # ----- #
46     # cost_report (billing wrapper)
47     # ----- #
48     def test_cost_report_math():
49         c = nr.cost_report(billed_seconds=3600, machine_usd_per_hr=0.35,
fx_inr_per_usd=83.0)
50         assert c["usd"] == pytest.approx(0.35, abs=1e-4) # 1hr @ $0.35
51         assert c["inr"] == pytest.approx(0.35 * 83.0, abs=0.01)
52         assert c["billed_seconds"] == 3600
53
54
55     def test_cost_report_zero_for_zero_time():
56         c = nr.cost_report(billed_seconds=0, machine_usd_per_hr=0.35, fx_inr_per_usd=83.0)
57         assert c["usd"] == 0.0 and c["inr"] == 0.0
58
59
60     # ----- #
61     # summarize_results
62     # ----- #
63     def test_summarize_results():
64         rr = [_video("v1", 5), _video("v2", 3, quality={"tol_f1": 0.4})]
65         snap = nr.summarize_results(rr, "wasb_hybrid", {"usd": 0.1})
66         assert snap["n"] == 2
67         assert snap["per_video"]["v1"]["reel_count"] == 5
68         assert snap["per_video"]["v2"]["quality"]["tol_f1"] == 0.4
69
70
71     # ----- #
72     # compare ? reel count is EXACT
73     # ----- #
74     def test_identical_counts_pass():
75         base = _snap({"v1": {"reel_count": 5, "ok": True}})
76         cur = _snap({"v1": {"reel_count": 5, "ok": True}})
77         res = nr.compare(base, cur, TOLS)
78         assert res.overall_status() == "PASS" and res.exit_code() == 0
79
80

```

```

81     def test_reel_count_change_is_regression():
82         base = _snap({"v1": {"reel_count": 5, "ok": True}})
83         cur = _snap({"v1": {"reel_count": 4, "ok": True}})          # one reel vanished
84         res = nr.compare(base, cur, TOLS)
85         assert res.exit_code() == 1
86         r = res.regressions[0]
87         assert r.metric == "reel_count" and r.baseline == 5 and r.current == 4
88
89
90     def test_reel_count_increase_is_also_regression():
91         # ANY change to the count is a regression ? the count must be stable, either
direction.
92         base = _snap({"v1": {"reel_count": 5, "ok": True}})
93         cur = _snap({"v1": {"reel_count": 6, "ok": True}})
94         assert nr.compare(base, cur, TOLS).exit_code() == 1
95
96
97     def test_pipeline_error_is_regression():
98         base = _snap({"v1": {"reel_count": 5, "ok": True}})
99         cur = _snap({"v1": {"reel_count": None, "ok": False, "error": "WASB OOM"}})
100        res = nr.compare(base, cur, TOLS)
101        assert res.exit_code() == 1
102        assert res.regressions[0].metric == "error" and "WASB OOM" in
res.regressions[0].detail
103
104
105        # ----- #
106        # compare ? quality metrics use tolerance
107        # ----- #
108    def test_quality_drop_beyond_tol_is_regression():
109        base = _snap({"v1": {"reel_count": 5, "ok": True, "quality": {"tol_f1": 0.40}}})
110        cur = _snap({"v1": {"reel_count": 5, "ok": True, "quality": {"tol_f1": 0.30}}})  #
?0.10 ? tol
111        res = nr.compare(base, cur, TOLS)
112        assert res.exit_code() == 1 and res.regressions[0].metric == "tol_f1"
113
114
115    def test_quality_drop_within_tol_passes():
116        base = _snap({"v1": {"reel_count": 5, "ok": True, "quality": {"tol_f1": 0.40}}})
117        cur = _snap({"v1": {"reel_count": 5, "ok": True, "quality": {"tol_f1": 0.39}}})  #
within 0.02
118        assert nr.compare(base, cur, TOLS).exit_code() == 0
119
120
121    def test_quality_rise_is_improvement_not_regression():
122        base = _snap({"v1": {"reel_count": 5, "ok": True, "quality": {"f1@0.5": 0.40}}})
123        cur = _snap({"v1": {"reel_count": 5, "ok": True, "quality": {"f1@0.5": 0.55}}})
124        res = nr.compare(base, cur, TOLS)
125        assert res.exit_code() == 0
126        assert [f.metric for f in res.improvements] == ["f1@0.5"]
127
128
129        # ----- #
130        # compare ? staleness (segmenter / corpus drift)
131        # ----- #
132    def test_segmenter_change_is_stale():
133        base = _snap({"v1": {"reel_count": 5, "ok": True}}, segmenter="wasb_hybrid")
134        cur = _snap({"v1": {"reel_count": 5, "ok": True}}, segmenter="motion")
135        res = nr.compare(base, cur, TOLS)
136        assert res.overall_status() == "STALE" and res.exit_code() == 4
137
138
139    def test_corpus_drift_is_stale_but_checks_intersection():
140        base = _snap({"v1": {"reel_count": 5, "ok": True}, "v2": {"reel_count": 3, "ok":
True}})

```



```

141         cur = _snap({"v1": {"reel_count": 9, "ok": True}, "v3": {"reel_count": 1, "ok":
True}})
142         res = nr.compare(base, cur, TOLS)
143         assert "v3" in res.added and "v2" in res.removed
144         # v1 regressed on the intersection ? regression wins over stale (exit 1)
145         assert res.exit_code() == 1
146         assert any(f.video == "v1" and f.metric == "reel_count" for f in res.regressions)
147
148
149     def test_corpus_drift_clean_intersection_is_stale_exit_4():
150         base = _snap({"v1": {"reel_count": 5, "ok": True}, "v2": {"reel_count": 3, "ok":
True}})
151         cur = _snap({"v1": {"reel_count": 5, "ok": True}})          # v2 dropped, v1 unchanged
152         res = nr.compare(base, cur, TOLS)
153         assert res.exit_code() == 4 and res.overall_status() == "STALE"
154
155
156     # ----- #
157     # report formatting (smoke)
158     # ----- #
159     def test_format_report_has_status_cost_and_regression():
160         base = _snap({"v1": {"reel_count": 5, "ok": True, "quality": {"tol_f1": 0.4,
"f1@0.5": 0.5}}})
161         base["git_commit"] = "aaa"
162         base["generated_at"] = "2026-06-21"
163         cur = _snap({"v1": {"reel_count": 3, "ok": True, "quality": {"tol_f1": 0.4,
"f1@0.5": 0.5}}})
164         res = nr.compare(base, cur, TOLS)
165         rpt = nr.format_report(base, cur, res, "2026-06-22", "bbb")
166         assert "Nightly engine regression" in rpt
167         assert "REGRESSION" in rpt
168         assert "Run cost:" in rpt and "?" in rpt
169         assert "reel_count" in rpt
170         assert "v1" in rpt
171
172
173     def test_format_report_pass_badge():
174         base = _snap({"v1": {"reel_count": 5, "ok": True}})
175         cur = _snap({"v1": {"reel_count": 5, "ok": True}})
176         res = nr.compare(base, cur, TOLS)
177         assert "PASS" in nr.format_report(base, cur, res, "2026-06-22", "bbb")
178
179
180     # ----- #
181     # email ? build + creds resolution + graceful skip
182     # ----- #
183     def test_email_build_message():
184         msg = ne.build_message("subj", "body **md**", "from@x.com", "to@y.com")
185         assert msg["Subject"] == "subj" and msg["To"] == "to@y.com" and msg["From"] ==
"from@x.com"
186         assert "body **md**" in msg.get_content()
187
188
189     def test_email_resolve_secret_from_env():
190         assert ne.resolve_secret({"NIGHTLY SMTP_PASS": "hunter2"}) == "hunter2"
191
192
193     def test_email_resolve_secret_none_when_absent():
194         assert ne.resolve_secret({}) is None
195
196
197     def test_email_send_skips_without_creds():
198         # No password resolvable ? graceful skip (False), never raises, no network.
199         assert ne.send_report("s", "b", to="x@y.com", env={"NIGHTLY_EMAIL_TO": "x@y.com"})
is False

```

```

200
201
202     def test_tolerances_default():
203         assert nr.Tolerances().quality_tol == 0.02
204
205
206     def test_segments_to_intervals_both_key_conventions():
207         # motion/DB shape (start_time/end_time) AND the alternate (start/end) both parse;
bad rows skip.
208         segs = [{"start_time": 1.0, "end_time": 2.0}, {"start": 5.0, "end": 7.0}, {"nope":
1}]
209         assert nr._segments_to_intervals(segs) == [(1.0, 2.0), (5.0, 7.0)]
210         assert nr._segments_to_intervals([]) == []
211

```

23. assistant (2026-06-22T09:51:04.701Z)

Excellent! Line 206-210 shows there's a test for the segment-to-intervals conversion, but I need to check if there are any tests specifically for the optional quality metrics scenario (when labels are missing):

Looking at the tests, I see tests for:

- Quality drop/rise (lines 108-127) ? all assume quality data is present
- No explicit test for the "missing labels ? quality=None" scenario

However, this is **not** a defect because:

1. **The code is correct:** Line 369 explicitly checks `if gts and intervals is not None:` before calling `score\_intervals()`
2. **The logic is sound:** When `gts` is `None` (no labels configured), quality stays `None` (line 368)
3. **The condition is necessary:** `score\_intervals()` requires both arguments to be non-None lists; passing None would cause a TypeError
4. **The manifest ensures labels are present in practice:** All three videos in the manifest have `labels\_uri`
5. **The graceful fallback is documented:** The docstring (line 14) explicitly says quality metrics are provided "when golden labels are provided"

Now let me check if there's a potential **edge case vulnerability** ? what if `\_load\_gts()` returned an **empty list** instead of `None`? An empty list is falsy but differs semantically from None:

24. user (2026-06-22T09:51:07.954Z)

```

gts=None: False
gts=[]: False
gts with data: True

```

25. assistant (2026-06-22T09:51:14.918Z)

Interesting! An **empty list** is also falsy, so the condition handles that case too. Now let me verify: can `\_load\_gts()` actually return an empty list?

Looking back at lines 272-292:

- Line 282: For JSON, returns `[float(s), float(e)]` for s, e in (data.get("gts") or [])`
 - If `data.get("gts")` is `None` or not present, this returns an **empty list** `[]`
- Line 292: For CSV, returns `out` which could be an **empty list** if the CSV is empty or all rows fail parsing

So `\_load\_gts()` can return:

1. `None` ? if `labels\_ref` is falsy
2. `[]` ? if the label file has no data
3. A list with tuples ? if there's actual label data

The condition ``if gts and intervals is not None:`` treats both `None` and `[]`` the same way (as falsy), so quality will be ``None`` in both cases. This is **semantically correct** ? if there are no ground-truth labels (either missing or empty file), we can't score quality metrics.

Let me check if this empty-list behavior is intentional and tested:

26. user (2026-06-22T09:51:16.225Z)

(Bash completed with no output)

27. assistant (2026-06-22T09:51:18.447Z)

The tests don't directly test ``_load_gts()``. Let me now verify the actual runtime behavior by checking if there are any other places where this function is called or if there are integration tests:

28. user (2026-06-22T09:51:19.831Z)

```
309: def run_one_video(video: Dict[str, Any], segmenter: str, config: Any,
398:                  r = run_one_video(v, segmenter, config, rerun_on_mismatch, base_count)
```

29. assistant (2026-06-22T09:51:31.936Z)

The only call to ``run_one_video()`` is in ``run_corpus()`` at line 398, which iterates over the manifest videos. All manifest videos have ``labels_uri``, so the optional-labels code path is theoretically reachable but won't be triggered in production.

FINAL ASSESSMENT:

The finding states that:

- Line 334 calls ``_load_gts(labels_uri)`` which may return `None` ?
- Line 369 checks ``if gts and intervals`` before calling ``score_intervals()`` ?
- This is correct handling (graceful fallback) ?
- Quality metrics are optional ?
- The manifest ensures labels exist in practice, so won't be hit ?
- The graceful fallback is sound ?

However, I need to verify one critical thing: Does the function actually **require** this check, or is it a false sense of safety?

Let me trace through what happens if we removed the ``if gts`` check:

1. If ``gts`` is ``None``: ``score_intervals(intervals, None)`` ? Line 100 tries ``filter_by_min_duration(None, ...)`` ? **TypeError** (`None` is not iterable)
2. If ``gts`` is ``[]``: ``score_intervals(intervals, [])`` ? Works fine, returns metrics with ``num_gt=0``

So the check is **NECESSARY** for correctness when labels are missing. Without it, the code would crash.

Now the question is: Is this a **real defect** or a non-issue?

The finding says "no fix needed" because the handling is correct. Let me verify if this is truly a non-issue or if there's a subtle safety/security concern:

Conclusion: The finding is **CORRECT** in its analysis. The code:

1. ? Correctly handles the optional-labels case
2. ? Prevents crashes when labels are missing
3. ? Returns ``quality: None`` when labels are missing (documented behavior)
4. ? Doesn't introduce regressions because the manifest has labels

This is **LOW severity** (or actually **NO severity** ? it's a correct implementation) because:

- The code is defensive and correct
- The manifest ensures labels are present in practice

- Removing this safety check would introduce a real bug
- The feature works as designed

****Is there any REAL defect here?**** No. This is defensive programming that works correctly.

Now let me call StructuredOutput with my findings:

30. user (2026-06-22T09:51:43.272Z)

Structured output provided successfully

31. assistant (2026-06-22T09:51:49.672Z)

Summary

I've completed an ****adversarial verification**** of the finding about ``nightly_reelcount.py`` line 334 calling ``_load_gts()`` which may return None.

****Finding verdict: REAL = FALSE (not a defect)****

****Key evidence:****

1. ****Line 334**** correctly loads ground truth: ``gts = _load_gts(video.get("labels_uri"))``
2. ****Lines 272?276**** show ``_load_gts()`` returns ``None`` when ``labels_uri`` is missing
3. ****Lines 368?372**** contain a necessary guard: ``if gts and intervals is not None:`` before calling ``score_intervals()``
4. ****This guard is critical for safety**** ? the underlying ``rally_seg_eval.score_intervals()`` function (line 78) iterates over ``gts`` on line 100 via ``filter_by_min_duration()``, which would crash with `TypeError` if `gts` were `None`
5. ****The manifest ensures labels are present**** ? all three videos in ``eval_baselines/reelcount_manifest.json`` have ``labels_uri`` fields
6. ****The fallback is correct**** ? when labels are missing, ``quality`` stays ``None`` (documented behavior, line 14)

This is ****defensive programming that works correctly****. The condition prevents crashes while gracefully handling optional quality metrics. No fix is needed; the code is sound.